

CYBER SENTINEL

SYSTEM LEVEL V2.0 PRO SPECIFICATIONS

REAL-TIME CYBER THREAT INTELLIGENCE FEED,
Nation-State APT Actor Profiling,
Mitigation Center CVE Registry, & Firewall Policy Generator.

SECURE OPERATION CENTER ACCESS LOG

CODENAME: CYBER-SENTINEL-CTI
RELEASE STAGE: TECHNICAL MANUAL
COMPILE TIME: JUNE 18, 2026

SECTION 1.0 // SYSTEM OVERVIEW & DESIGN ARCHITECTURE

The Cyber Sentinel platform is a unified cyber threat intelligence (CTI) dashboard and security operation command interface. The system acts as a real-time monitor for global network indicators of compromise (IoC) combined with threat actor profile catalogs, remediation lookup sheets, and copy-pastable firewall rules compiling.

Design Architecture relies on a split-client system that maps real-time geolocation indicators directly onto a customized vector map, processing background intelligence datasets, and generating firewall rule structures dynamically.

The technology stack comprises the following key components:

- * **FastAPI Backend:** Lightweight Python 3 web service serving simulated real-time data payloads and routing lookup engines.
- * **React 19 Frontend:** Page Application (SPA) dashboard containing responsive widgets, tabs state, and user actions.
- * **Leaflet Maps API:** Global geographic vectors utilizing customized tile maps, animation loops, and popups.
- * **Recharts Engines:** Uses attack vectors to output categorized distributions in clean SVG layouts.

SECTION 2.0 // CORE DIRECTORY MAP

The workspace structure contains clean decoupling between the client assets and server services:

```
cyber-threat-intelligence/
|-- start.sh                # Unified concurrent launch script
|-- backend/
|   |-- main.py             # FastAPI server, routers, & datasets
|   |-- requirements.txt    # Python packages (fastapi, uvicorn)
|   +-- venv/               # Python 3 virtual environment
+-- frontend/
|   |-- .env                # Vite environment configuration file
|   |-- index.html          # Entry meta tags and html structure
|   |-- src/
|       |-- main.jsx        # React mounting structure
|       |-- App.jsx         # Dashboard module layout, maps, & state
|       +-- index.css       # Theme imports, modals, & glow styles
+-- package.json            # Client packages and build commands
```

SECTION 3.0 // OPERATIONAL FEATURES MANUAL

Cyber Sentinel V2.0 provides an interactive operations command console structured across four specialized dashboard sections:

- * **Live Threat Monitor:** visual module displaying origin-to-target attack vectors as dashed lines. Glowing markers fly along coordinates dynamically based on interpolation loops, triggering concentric expanding rings (shockwaves) upon collision with target countries. It integrates a live threat log feed with severity indicators, IP reputation scanning, and attack category distributions.
- * **Simulator Control:** top header integrates speed multipliers (1x, 2x, 5x) and play/pause controls to speed up or pause the background threat stream feed simulator. You can inspect threats in details or export feed datasets directly as JSON or CSV files.
- * **APT Threat Actors:** on-state adversary catalog. Groups like Lazarus Group (APT38), Cozy Bear (APT29), Fancy Bear (APT28), Volt Typhoon, and Sandworm are profiled with threat levels, origins, target industries, preferred vectors, and CVE exploit history.
- * **Mitigation CVEs:** on Vulnerabilities and Exposures (CVE) search engine. Displays critical vulnerabilities with CVSS scores, threat details, and explicit patch recommendations or firewall blockings.
- * **Defensive Toolkit:** dynamically compiles IP blocking shell rules. Users check suspicious IPs in the logs to generate output block scripts for UFW (deny host), iptables (INPUT DROP), or Cisco ASA (access-list deny).

SECTION 4.0 // DEFCON ALARM STATES

The DEFCON level evaluates the percentage of high-severity alerts in your sensor history and changes state dynamically:

- * **DEFCON 1 (Red):** 40% high risk threat alerts. Critical Alert. Severe active threat status. High alerts active.
- * **DEFCON 2 (Purple):** 40% high risk threat alerts. Elevated Actor Activity. Targeted campaigns active.
- * **DEFCON 3 (Yellow):** 25% high risk threat alerts. Elevated System Alert. Moderate scanner activity.
- * **DEFCON 4 (Cyan):** 12% high risk threat alerts. Systems Operational. Normal background surveillance.

SECTION 5.0 // BACKEND API ROUTING SPECIFICATION

The backend FastAPI server runs on port 8000. It exposes REST API routes serving JSON structures to the frontend:

- * **GET /api/threats:** returns a JSON list of active threat payloads containing indicator IP, type, signature, severity, origin country, and timestamp.
- * **GET /api/map-data:** connecting flight pathways for the map layout containing geocoded origin and target coordinates, payload type, and values.
- * **GET /api/apt-groups:** the database registry of nation-state threat actors.
- * **GET /api/cves:** returns the vulnerability database registry with CVSS score metrics and mitigations.
- * **GET /api/check-ip/{ip}:** checks risk index, threat category, owner, and geolocation. The score is simulated in python via MD5 hashing on input IP.

SECTION 6.0 // API RESPONSE EXAMPLE

Example JSON output from GET /api/check-ip/8.8.8.8:

```
{
  "ip": "8.8.8.8",
  "risk_score": 76,
  "category": "High Risk",
  "threat_types": [
    "Botnet C2",
    "Malware Distribution"
  ],
  "isp": "Google Cloud",
  "country": "Brazil",
  "country_code": "BR",
  "latitude": -14.235,
  "longitude": -51.925,
  "last_active": "2026-06-18T06:05:00Z"
}
```

SECTION 7.0 // INSTALLATION & OPERATING MANUAL

Operating both servers concurrently is managed using the VITE environment files and the bash launcher. Set up the frontend to target the correct backend host.

Create a frontend/.env file to override the default API url target:

```
# frontend/.env
VITE_API_BASE_URL=http://localhost:8000
```

To start both servers with a single command from the project root directory, run:

```
chmod +x start.sh
./start.sh
```

This command executes the following script to spin up the servers concurrently and clean up cleanly on exit:

```
#!/bin/bash

# Function to kill child processes on exit
cleanup() {
    echo ""
    echo "Stopping servers..."
    kill $(jobs -p) 2>/dev/null
}
trap cleanup EXIT

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# Start backend
echo "Starting Backend API (Uvicorn)..."
cd "$PROJECT_ROOT/backend"
./venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000 --reload &

# Start frontend
echo "Starting Frontend App (Vite)..."
cd "$PROJECT_ROOT/frontend"
npm run dev &

wait
```

To terminate the servers, simply press CTRL+C once. The trap signal handler catches the interrupt and sends SIGTERM to both the Node.js (Vite) and Python (Uvicorn) background tasks.